

Coding Assignment for 9 to 10+ Years Experience

Mini Enterprise Helpdesk Ticket System
ASP.NET Core MVC/Web API + EF Core + Identity/JWT

Total Marks Allotted	100
Estimated Duration	16 to 24 Hours
Target Experience	9 to 10+ Years ASP.NET Core Developer
Project Title	Enterprise Helpdesk Ticket Management System in ASP.NET Core Clean Architecture

Instructions

- Read the complete assignment and plan the implementation before starting development.
- Use ASP.NET Core 6 or later. ASP.NET Core 8 is preferred.
- Use SQL Server with EF Core Code First migrations.
- Implement the project using MVC or Web API. A combined MVC + API approach is acceptable.
- Use proper naming conventions, comments where required, and a clean folder structure.
- Submit the source code with README.md, database migration/scripts, and setup steps.
- Make suitable assumptions and document them inside README.md.

Aim

To test the candidate capability in ASP.NET Core development using authentication, authorization, CRUD operations, EF Core, AJAX/API integration, validation, error handling, database relationships, dashboard, notifications, logging, testing basics, and deployment readiness.

Why this project is selected

- This single small project shows ASP.NET Core MVC/Web API knowledge.
- It covers EF Core, SQL design, Identity/JWT, validation, security, logging, and dashboard concepts.
- It gives practical exposure to real enterprise ticket/helpdesk workflows.
- It is suitable for 9 to 10+ years experience and demonstrates senior-level architecture thinking.

Task 1: Login / Register

Description: Implement user authentication using ASP.NET Core Identity or JWT authentication.

- Create Register page/API for new users.
- Create Login page/API for existing users.
- Create three roles: Admin, Support, User.
- Assign default role User during registration.
- Seed Admin and Support users during application startup or database seeding.
- Use secure password hashing provided by ASP.NET Core Identity.
- Redirect user to Dashboard after successful login.

Task 2: Role-Based Authorization

Description: Restrict screens and APIs based on user role.

- Admin can manage all tickets and assign tickets to Support users.
- Support can view assigned tickets and change ticket status.
- User can create tickets and view only their own tickets.
- Apply [Authorize] and role-based policies on controllers/actions.
- Hide or show UI buttons according to logged-in user role.

Task 3: Ticket Management

Description: Implement complete ticket workflow with Create, Assign, Status Change, Comments, and Attachment upload.

Ticket Status Values

- Open
- In Progress
- Resolved
- Closed

Ticket Priority Values

- Low
- Medium
- High
- Critical

Required Ticket Fields

- Title - required, maximum 150 characters.
- Description - required, maximum 2000 characters.
- Priority - required dropdown.
- Status - default Open.
- CreatedByUserId - logged-in user.
- AssignedToUserId - nullable, selected by Admin.
- CreatedOn and UpdatedOn date-time fields.
- IsDeleted for soft delete.

Task 4: Ticket CRUD Operations

Create Ticket

- Create ticket form using Razor page/view or API form.
- Validate Title, Description, and Priority on client side and server side.
- Save ticket in SQL Server using EF Core.
- Default status should be Open.
- Store CreatedByUserId and CreatedOn automatically.

View Tickets

- Display tickets in a table.
- User should see own tickets only.
- Support should see assigned tickets.
- Admin should see all tickets.
- Show Title, Priority, Status, Created By, Assigned To, Created Date, and Action buttons.

Update Ticket

- Edit Title, Description, and Priority where allowed.
- Admin can assign ticket to Support user.
- Support can update status from Open to In Progress, Resolved, or Closed.
- Validate ModelState before saving.
- Update UpdatedOn automatically.

Delete Ticket

- Use soft delete instead of permanent delete.
- Show confirmation dialog before delete.
- Only Admin or ticket owner can delete as per project assumption.
- Deleted tickets should not appear in normal list.

Task 5: Comments and Attachment Upload

- Allow users to add comments on tickets.
- Store comment text, TicketId, CreatedByUserId, and CreatedOn.
- Allow attachment upload for ticket supporting files.
- Restrict attachment types to PDF, JPG, PNG, DOCX, XLSX, or ZIP.
- Restrict maximum file size, for example 5 MB.
- Store file path/name in TicketAttachments table.

Task 6: Dashboard

Description: Create a dashboard after login to show ticket summary.

- Total Tickets count.
- Pending/Open Tickets count.
- Resolved Tickets count.
- Tickets by Priority count.
- Tickets by Status chart.
- Recent Tickets list.
- Role-wise dashboard data: Admin sees all, Support sees assigned, User sees own.

Task 7: Search and Server-Side Pagination

- Implement search by Title.
- Implement filter by Status.
- Implement filter by Priority.
- Implement server-side pagination using page number and page size.
- Implement sorting by Created Date or Priority.
- Return paginated response model for API or partial view for MVC.

Task 8: Audit Trail

Description: Maintain history of important ticket actions.

- Track who created the ticket.
- Track who assigned the ticket.
- Track who changed the ticket status.
- Track old status and new status.
- Track action date and time.
- Store logs in TicketAuditLogs table.
- Display audit history on ticket detail page.

Task 9: Notifications

- Send notification when a ticket is assigned.
- Send notification when ticket status is changed.
- Use Email notification or SignalR real-time notification.
- Store notification records in database.
- Show read/unread notification count to logged-in user.

Task 10: Security Requirements

- Use role-based authorization.
- Use input validation for all forms/API requests.
- Use global exception handling middleware.
- Use secure password hashing via ASP.NET Core Identity.
- Use CSRF protection for MVC forms.
- Use JWT protection for API endpoints if Web API approach is selected.
- Do not expose sensitive exception details in Production.
- Validate uploaded file type and size.

Task 11: Database and EF Core Code First

- Use SQL Server as database.
- Use EF Core Code First migrations.
- Create proper primary key and foreign key relationships.
- Create indexes on Status, Priority, CreatedByUserId, AssignedToUserId, and CreatedOn.
- Implement soft delete using IsDeleted field.
- Use CreatedOn, CreatedBy, UpdatedOn, UpdatedBy fields where required.

Recommended Database Tables

- AspNetUsers / ApplicationUser
- AspNetRoles
- Tickets
- TicketComments
- TicketAttachments
- TicketAuditLogs
- Notifications

Important Relationships

- Ticket.CreatedByUserId -> ApplicationUser.Id
- Ticket.AssignedToUserId -> ApplicationUser.Id
- TicketComment.TicketId -> Ticket.Id
- TicketAttachment.TicketId -> Ticket.Id
- TicketAuditLog.TicketId -> Ticket.Id
- Notification.UserId -> ApplicationUser.Id

Task 12: Architecture Requirements

Description: Implement Clean Architecture with proper separation of concerns.

- Helpdesk.Api - Controllers, API endpoints, middleware registration, Swagger.
- Helpdesk.Application - DTOs, interfaces, services, validation logic.
- Helpdesk.Domain - Entities, enums, common base classes.
- Helpdesk.Infrastructure - EF Core DbContext, repositories, Identity, email service.
- Helpdesk.Tests - Unit tests and basic integration tests.

Task 13: DevOps and Documentation

- Create Dockerfile for ASP.NET Core application.
- Create docker-compose.yml for application and SQL Server container if possible.
- Provide appsettings.Development.json sample.
- Provide README.md with setup steps.
- Enable Swagger API documentation.
- Document assumptions, roles, credentials, and database migration commands.
- Push code to GitHub and share repository link.

Task 14: Testing and Quality Assurance

- Test Login and Register functionality.
- Test role-wise access: Admin, Support, User.
- Test Create, Update, View, Delete ticket functionality.
- Test ticket assignment workflow.
- Test status change workflow.
- Test comment and attachment upload.
- Test dashboard counts.
- Test search and pagination.
- Test validation and error handling.
- Test notification generation.

Expected Learning/Assessment Areas

- ASP.NET Core MVC/Web API implementation.
- EF Core and SQL Server database design.
- Identity/JWT authentication and authorization.
- Clean Architecture implementation.
- Security and validation practices.
- Logging and exception handling.
- Testing and documentation basics.
- Real enterprise thinking through ticket workflow.

Candidate Checklist

Complete the Status column and submit this checklist along with source code.

Task No.	Task / Description	Subtasks	Assessment Criteria	Status
1	Login / Register	Register, Login, Identity/JWT, seed roles Admin/Support/User.	User can register/login and roles are created correctly.	
2	Role Authorization	Admin, Support, User permissions on UI and APIs.	Each role can access only allowed features.	
3	Ticket Management	Create, assign, status change, close ticket.	Ticket workflow works as per role.	
4	Comments & Attachments	Add comments and upload ticket files.	Comments and files are saved and displayed correctly.	
5	Dashboard	Total, pending, resolved, priority-wise counts.	Dashboard counts are correct role-wise.	
6	Search & Pagination	Search by title/status/priority with server-side paging.	Filtering and pagination work correctly.	
7	Audit Trail	Track created by, assigned by, status changed by, date/time.	Ticket history is stored and visible.	
8	Notifications	Email or SignalR notification on assignment/status change.	Notification generated for correct user.	
9	Security	Role auth, validation, exception handling, CSRF/JWT, file validation.	Security checks implemented properly.	
10	Database	SQL Server, EF Core Code First, relationships, indexes, soft delete.	Database schema and relationships are correct.	
11	Architecture	Helpdesk.Api, Application, Domain, Infrastructure, Tests.	Project follows Clean Architecture structure.	
12	DevOps & Docs	Dockerfile, docker-compose, appsettings, README, Swagger.	Application can be setup using documented steps.	

List Assumptions

- Candidate may choose MVC, Web API, or a hybrid approach.
- For 9 to 10+ years experience, proper Clean Architecture with clear separation of concerns is expected.
- Email notification may be simulated/logged if SMTP is not available.
- SignalR is optional if email notification is implemented.
- Docker compose may be submitted as optional if local SQL Server is used.

Submission Requirements

- GitHub repository link.
- Zip file if required.
- README.md with setup steps.
- Database migration commands or SQL script.
- Screenshots of main pages.
- Completed checklist.